Solution: Richest Customer Wealth (easy)

Problem Statement

Given a 2D array `accounts` of `m x n` size, `accounts[j]` represents money in the `jth` bank account of the `ith` customer.

Return the wealth of the richest individual from a set of bank customers.

Imagine every customer has multiple bank accounts, with each account holding a certain amount of money. The total wealth of a customer is calculated by summing all the money across their multiple accounts.

Examples

1. **Example 1**:
   - **Input**: `[[5,2,3],[0,6,7]]`
   - **Expected Output**: `13`
   - **Justification**: The total wealth of the first customer is 10 and of the second customer is 13. So, the output is 13 as it's the maximum among all customers.
2. **Example 2**:
   - **Input**: `[[1,2],[3,4],[5,6]]`
   - **Expected Output**: `11`
   - **Justification**: Total wealth for each customer is [3, 7, 11]. Maximum of these is 11.
3. **Example 3**:
   - **Input**: `[[10,100],[200,20],[30,300]]`
   - **Expected Output**: `330`
   - **Justification**: Total wealth for each customer is [110, 220, 330]. The wealthiest customer has 330.

Solution

The algorithm aims to traverse each customer's accounts, compute their total wealth by summing up the balances, and track the wealthiest customer found so far. We utilize a straightforward iterative method, whereby we loop through each customer and their respective accounts, calculating the sum of each customer's accounts, and then making a comparison against a stored maximum wealth variable. If the current customer's total wealth surpasses our stored maximum, we update our stored value. Upon completion of the iteration through all customers, our stored maximum wealth value represents the wealth of the richest customer. Simplicity and a single-pass solution make this algorithm appealing, ensuring computational efficiency by avoiding nested loops and minimizing computational steps where possible.

Detailed Steps:

Step 1: Initialize a variable `maxWealth` to store and track the maximum wealth found during the iteration. Set its initial value to 0 as we will use it for comparison.

Step 2: Iterate through the 2D-array of accounts using a loop. Each sub-array represents one customer's accounts.

Step 3: For each customer, calculate their total wealth by summing up all the values in their respective sub-array. Utilize another loop or a sum function for this.

Step 4: Compare the computed sum of the current customer with `maxWealth`. If it's greater, update `maxWealth` with the new value.

Step 5: Repeat steps 3-4 for all customers in the array.

Step 6: Once the iteration is complete, `maxWealth` holds the maximum wealth among all customers. Return this value as the output.

Algorithm Walkthrough

Let's illustrate the algorithm using a simple example with an input array `[[5,10,15],[10,20,30],[15,30,45]]`:

- **Initialize `maxWealth` to 0**: This will store the maximum wealth we find as we traverse through the accounts.

  `maxWealth = 0`
- **Iterating through the customer arrays**:
  - First customer: `[5,10,15]`
    - Calculate total wealth: `5 + 10 + 15 = 30`
    - Compare and update `maxWealth`: `maxWealth` (0) < `30` => Update `maxWealth` to `30`
  - Second customer: `[10,20,30]`
    - Calculate total wealth: `10 + 20 + 30 = 60`
    - Compare and update `maxWealth`: `maxWealth` (30) < `60` => Update `maxWealth` to `60`
  - Third customer: `[15,30,45]`
    - Calculate total wealth: `15 + 30 + 45 = 90`
    - Compare and update `maxWealth`: `maxWealth` (60) < `90` => Update `maxWealth` to `90`
- **Conclusion**: The final value of `maxWealth` is `90`, which represents the richest customer's wealth. So, `90` is returned as the output.

This approach guarantees we evaluate the total wealth of each customer and always maintain the wealthiest customer's wealth encountered thus far in our `maxWealth` variable, ensuring we find the correct answer by the end of our iterations.

Here is the visual representation of the algorithm:

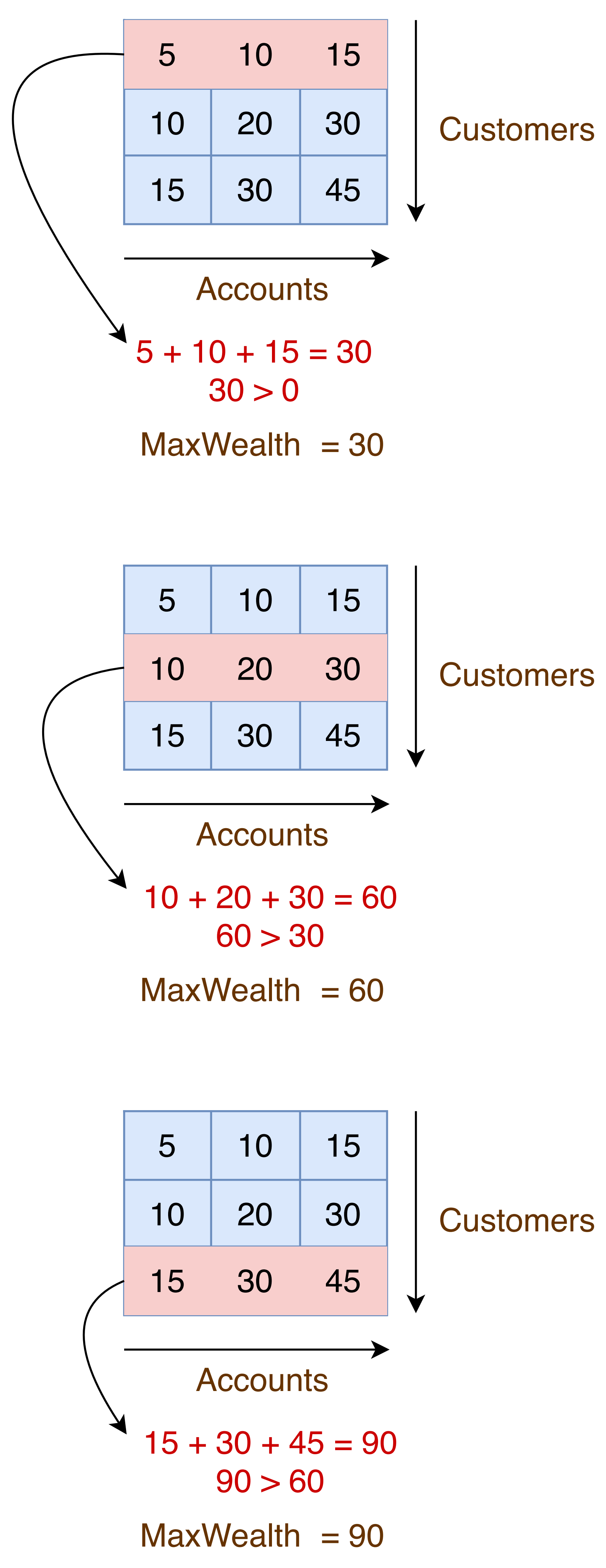
Example 1:

Input: `[[5,10,15],[10,20,30],[15,30,45]]`

① Initialize MaxWealth to store the maximum wealth.

MaxWealth = 0

② Iterate through the customer arrays to find the max. wealth in a account.

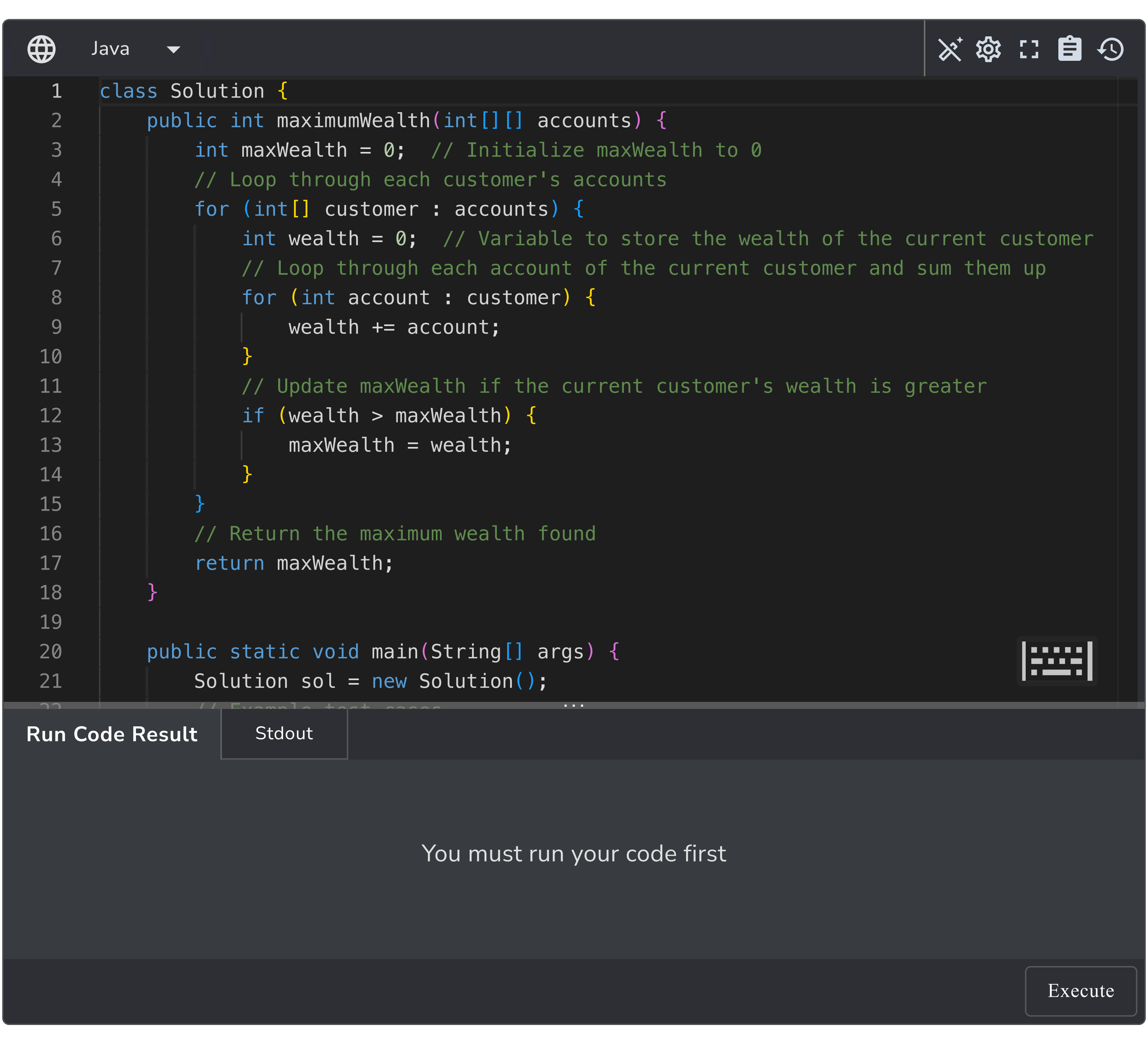


Output: Richest Customer Wealth = 90

Richest Customer Wealth

Code

Here is the code for this algorithm:



```java
class Solution {
    public int maximumWealth(int[][] accounts) {
        int maxWealth = 0;  // Initialize maxWealth to 0
        // Loop through each customer's accounts
        for (int[] customer : accounts) {
            int wealth = 0;  // Variable to store the wealth of the current customer
            // Loop through each account of the current customer and sum them up
            for (int account : customer) {
                wealth += account;
            }
            // Update maxWealth if the current customer's wealth is greater
            if (wealth > maxWealth) {
                maxWealth = wealth;
            }
        }
        // Return the maximum wealth found
        return maxWealth;
    }

    public static void main(String[] args) {
        Solution sol = new Solution();
```

Time Complexity: O(n*m)

- The algorithm traverses through all the customers and their respective accounts once.
- If `n` represents the number of customers and `m` represents the maximum number of accounts (i.e., length of sub-arrays), in the worst-case scenario, the algorithm would need to perform O(n*m) operations.
- Iterating through each customer is an O(n) operation, and calculating the sum of their wealth by iterating through each of their `m` accounts makes the inner operation O(m). Together, this results in a time complexity of O(n*m).

Space Complexity: O(1)

- The algorithm only utilizes a constant amount of extra space, which is not dependent on the input size. Specifically, it uses two additional variables: `maxWealth` and `wealth`, to keep track of the maximum wealth found so far and the wealth of the current customer, respectively.
- Since these variables are used for storing scalar values and their quantity does not grow with the input, the space complexity is O(1).